



INSTITUTO POLITÉCNICO DA UNIVERSIDADE KIMPA VITA

**(IP-UNIKIVI)**

LICENCIATURA EM ENGENHARIA INFORMÁTICA

**UNIDADE CURRICULAR: LINGUAGEM DE PROGRAMAÇÃO IV**

---

## **SISTEMA DE GESTÃO DE CLÍNICA MÉDICA**

**Por:**

**<Império Nakama/>**

IIº ANO

PERÍODO: TARDE

SALA: AUDITÓRIO II

O DOCENTE

---

ENGº. MOYO KANIVENGIDIO

UÍGE: 2025/2026

## **LISTA NOMINAL**

1-Amem Kiala Kahumba

2-António Chaves N. Monteiro

3- Augusto Quiatucana Alberto

4-Luís Manuel Marques

5-Pedro Kiamesso Rodrigues Nzongo

## INDÍCE

|                                               |   |
|-----------------------------------------------|---|
| 0. INTRODUÇÃO.....                            | 1 |
| 1. OBJECTIVOS .....                           | 2 |
| 1.1 Objectivo Geral.....                      | 2 |
| 1.2 Objectivo Específicos .....               | 2 |
| 2. PROGRAMAÇÃO ORIENTADA A OBJETOS.....       | 3 |
| 3 . LEVANTAMENTO E ANÁLISE DE REQUISITOS..... | 3 |
| 3.1 Requisitos Funcionais.....                | 3 |
| 3.1.1 Gestão de pacientes .....               | 3 |
| 3.1.2 Gestão de Médicos .....                 | 3 |
| 3.1.3 Gestão de Agendamento.....              | 3 |
| 3.1.3 Gestão de Prontuários.....              | 4 |
| 3.1.4 Controle de Acesso.....                 | 4 |
| 3.1.5 Relatórios.....                         | 4 |
| 3.2- Requisitos não Funcionais.....           | 4 |
| 4. TECNOLOGIAS UTILIZADAS .....               | 5 |
| 5 . ARQUITETURA DO SISTEMA.....               | 5 |
| 5.1- Implementação do Padrão MVC .....        | 5 |
| 5.2- Implementação do Padrão DAO .....        | 6 |
| 5.2.1- Operação CRUD Implementadas .....      | 7 |
| 6 -MODELAGEM DE BANCO DE DADOS .....          | 7 |
| 7 -INTERFACES DO SISTEMA.....                 | 9 |
| 8- CRONOGRAMA DE DESENVOLVIMENTO .....        | 1 |
| CONCLUSÃO.....                                | 1 |
| REFERÊNCIA BIBLIOGRÁFICA.....                 | 2 |

## **0. INTRODUÇÃO**

A gestão eficiente de clínicas médicas é essencial para garantir um atendimento rápido, seguro e organizado aos pacientes. Muitas clínicas ainda utilizam processos manuais ou sistemas isolados para controlar consultas, médicos e pacientes, e que pode causar erros, perda de informação e atrasos nos serviços. Com objectivo de solucionar esse problemas, foi desenvolvido um Sistema de Gestão de Clínica Médica utilizando assim a linguagem java. O sistema permite automatizar os processos administrativos, facilitando o cadastro de pacientes, médicos, consultas e pontuários médicos.

O projecto foi desenvolvido aplicando conceitos de programação Orientada a Objetos, interface gráfica, banco de dados relacional e padrões de arquitetura de software.

## **1. OBJETIVOS**

### **1.1 Objectivo Geral**

Desenvolver um sistema computacional capaz de gerenciar os processos de uma clínica médica de forma eficiente, segura e organizada.

### **1.2 Objectivo Específicos**

- ❖ Aplicar os conceitos de programação orientado a objetos;
- ❖ Implementar uma interface gráfica amigável;
- ❖ Integrar a aplicação ao banco de dados MySQL;
- ❖ Utilizar JDBC para acesso aos dados;
- ❖ Aplicar padrão MVC e DAO;
- ❖ Implementar autenticação de usuários;
- ❖ Gerar relatórios administrativos.

## **2. PROGRAMAÇÃO ORIENTADA A OBJETOS**

A Programação Orientada a Objetos (POO) é um paradigma de programação que organiza o desenvolvimento de software através da utilização de objetos, os quais representam entidades do mundo real e possuem características(atributos) e comportamentos(métodos). Este paradigma surgiu com o objetivo de tornar o sistemas mais organizados, reutilizáveis e fáceis de manter.

A Orientação Objetos baseia-se em quatro pilares fundamentais: Encapsulamento, Herança, Polimorfismo e Abstração. Esses conceitos permitem desenvolver sistemas mais seguros, flexíveis e escaláveis. (Shildt 2019)

## **3 . LEVANTAMENTO E ANÁLISE DE REQUISITOS**

A análise de requisitos constitui uma das etapas mais importante dos desenvolvimento de software, pois define claramente as necessidades dos utilizadores e as funcionalidades que o sistema deverá disponibilizar.

### **3.1 Requisitos Funcionais**

#### **3.1.2 Gestão de Médicos**

O sistema permite:

- ❖ Cadastro de médicos;
- ❖ Edição de dados;
- ❖ Exclusão de registros;
- ❖ Pesquisa por CRM ou nome.

#### **3.1.1 Gestão de pacientes**

O sistema permite:

- ❖ Cadastrar pacientes;
- ❖ Editar pacientes;
- ❖ Excluir paciente;
- ❖ Pesquisar pacientes por nome ou Bilhete de Identidade.

### 3.1.3 Gestão de Agendamento

O sistema permite:

- ❖ Marcação de consulta;
- ❖ Reagendamento;
- ❖ Cancelamento;
- ❖ Controle de horários.

### 3.1.3 Gestão de Prontuários

O sistema permite:

- ❖ Registrar diagnósticos;
- ❖ Registrar prescrições;
- ❖ Registrar observações clínicas.

### 3.1.4 Controle de Acesso

Perfis disponíveis:

- ❖ Administrador;
- ❖ Médico;
- ❖ Recepcionista.

### 3.1.5 Relatórios

- ❖ Consultas realizadas;
- ❖ Pacientes ativos;
- ❖ Consultas por médico.

## 3.2- Requisitos não Funcionais

Os requisitos não funcionais definem os padrões de qualidade que o sistema deve atender.

- ❖ Interface intuitiva;
- ❖ Segurança de dados;
- ❖ Boa performance;
- ❖ Facilidade de manutenção;
- ❖ Escalabilidade do sistema;

#### 4. TECNOLOGIAS UTILIZADAS

| TECNOLOGIAS | FUNÇÃO                     |
|-------------|----------------------------|
| Java        | Linguagem programação      |
| Java Swing  | Interface gráfica          |
| MySQL       | Banco de dados             |
| JDBC        | Conexão com banco de dados |
| MVC         | Arquitetura                |
| DAO         | Persistência de dados      |
| NetBeans    | ID para desenvolvimento    |
| GITHub      | Entrega do trabalho        |

#### 5. ARQUITETURA DO SISTEMA

O Sistema foi desenvolvido seguindo o padrão MVC (Model, View, Controller), permitindo a separação das responsabilidades entre interface gráfica, lógica de negócio e manipulação de dados. Para acesso ao banco de dados foi utilizado o padrão DAO, responsável por centralizar todas as operações JDBC e SQL. Essa organização torna o sistema mais modular, escalável e fácil de manter. (*Martin Fowler 2002*).

##### 5.1- Implementação do Padrão MVC

O padrão MVC (Model, View, Controller), garante a separação da interface, do controle de fluxo e da regra de negócio, em que cada tipo de componente executa um determinado tipo de tarefa. Essa separação permite alteração em cada uma das camadas isoladamente, diminuindo o trabalho da mudança constante das aplicações web. (*william Patatas Soares, 2006*).

No padrão MVC cada componente tem um tipo de responsabilidade:

- ❖ **Srvlets**: atuam como controlador (**controller**), que vão atender e entender as requisições dos usuários;
- ❖ **JavaBeans**: atuam como modelo (**model**), que representam e devem executar a regra de negócio;
- ❖ **JSP**: atuam como visão (**view**), ou seja, irão disponibilizar as diversas formas de visualização do modelo de dados. (*william Patatas Soares, 2006*).

Model: Responsável pelas entidades



- ❖ Paciente;
- ❖ Médico;
- ❖ Consulta;
- ❖ Prontuário;
- ❖ Usuário.

View: Responsável pelas interfaces gráficas

- ❖ Tela de login;
- ❖ Tela paciente;
- ❖ Tela médica;
- ❖ Tela consulta;
- ❖ Tela relatório.

Controller: Responsável pelo processamento das ações do usuário e comunicação entre view e model.

- ❖ Receber eventos da interface;
- ❖ Validar informações;
- ❖ Chamar métodos do model;
- ❖ Actualizar a interface.

## **5.2- Implementação do Padrão DAO**

O padrão DAO (Data Access Object) é uma solução amplamente utilizada no desenvolvimento de software para separar a lógica de acesso aos dados da lógica de negócio da aplicação. O principal objetivo deste padrão é criar uma camada específica responsável por realizar todas as operações relacionadas com persistência dos dados, independentemente da tecnologia de armazenamento utilizada.

A utilização do DAO permite abstrair os detalhes do banco de dados, possibilitando que as demais camadas do sistema trabalhem apenas com objetos do domínio da aplicação, sem necessidade de conhecer comandos SQL ou detalhes de implementação da base de dados.

No nosso Sistema de Gestão de Clínica Médica, o padrão DAO foi adotado para garantir uma arquitetura mais organizada, modular e de fácil manutenção. Cada entidade do sistema possui uma classe DAO responsável pelas operações de inserção, consulta,

atualização e exclusão de informação armazenadas no banco de dados MySQL. (*william Patatas Soares,2006*).

### 5.2.1- Operação CRUD Implementadas

As operações CRUD representam as funcionalidades básicas de manipulação de dados em sistema de informação.

Temos as seguintes operações:

- ❖ CREATE (Inserção): permite armazenar novos registros no banco de dados. Exemplo: cadastro de paciente, cadastro de médico e registro de consulta.
- ❖ READ (Consulta): permite recuperar informações armazenadas. Exemplo: pesquisa de paciente por nome, consulta de médico por especialidade e visualização de consultya agendadas.
- ❖ UPDATE (Atualização): permite modificar informações existente.Exemplo: alterar dados dom paciente, atualização de horário de uma consulta e modificação de informações de contacto.
- ❖ DELETE(Remoção): permite excluir ou inativar registros. Exemplo: exclusão de paciente inativos, cancelamento de consulta e remoção de utilizadores. (*william Patatas Soares,2006*).

Foram criadas as seguintes classes:

- ❖ ConsultaDAO.java
- ❖ MédicoDAO.java
- ❖ PacienteDAO.java
- ❖ PontuarioDAO.java
- ❖ UsuarioDAO.java

## 6 MODELAGEM DE BANCO DE DADOS

O banco de dados foi modelado utilizando o modelo Entidade Relacionamento (MER).

Entidade principais:

- ❖ Pacientes;
- ❖ Medicos;
- ❖ Consultas;
- ❖ Prontuarios;

❖ Usuário.

```
Script Base de Dados Clínica* x
Limit to 1000 rows

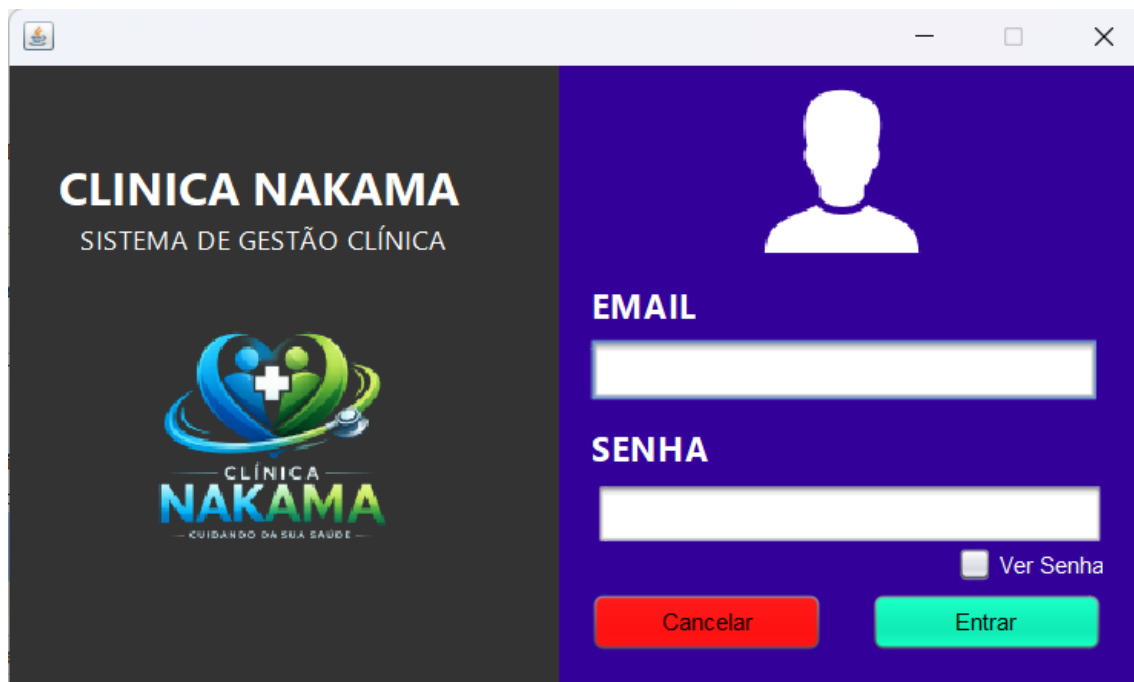
1 • create database clinicadb;
2 • CREATE TABLE pacientes (
3     id INT AUTO_INCREMENT PRIMARY KEY,
4     nome VARCHAR(150) NOT NULL,
5     bi VARCHAR(14),
6     nascimento DATE,
7     telefone VARCHAR(20),
8     endereco TEXT,
9     historico TEXT,
10    estado BOOLEAN DEFAULT TRUE
11 );
12 • CREATE TABLE medicos (
13     id INT AUTO_INCREMENT PRIMARY KEY,
14     nome VARCHAR(150) NOT NULL,
15     crm VARCHAR(50) UNIQUE,
16     especialidade VARCHAR(100),
17     telefone VARCHAR(20),
18     email VARCHAR(100),
19     estado BOOLEAN DEFAULT TRUE
20 );
21 • CREATE TABLE consultas (
22     id INT AUTO_INCREMENT PRIMARY KEY,
23     paciente_id INT NOT NULL,
24     medico_id INT NOT NULL,
25     data_consulta DATE NOT NULL,
26     hora_consulta TIME NOT NULL,
27     status ENUM('AGENDADA', 'CONFIRMADA', 'REALIZADA', 'CANCELADA') DEFAULT 'AGENDADA',
28     FOREIGN KEY (paciente_id) REFERENCES pacientes(id),
29     FOREIGN KEY (medico_id) REFERENCES medicos(id)
30 );
31 • CREATE TABLE prontuarios (
32     id INT AUTO_INCREMENT PRIMARY KEY,
33     consulta_id INT NOT NULL,
34     diagnostico TEXT,
35     medicamentos TEXT,
36     observacoes TEXT,
37     FOREIGN KEY (consulta_id) REFERENCES consultas(id)
38 );
39 • CREATE TABLE usuarios (
40     id INT AUTO_INCREMENT PRIMARY KEY,
41     nome VARCHAR(100) NOT NULL,
42     usuario VARCHAR(50) NOT NULL UNIQUE,
43     senha VARCHAR(255) NOT NULL,
44     perfil ENUM('ADMIN', 'MEDICO', 'RECEPCIONISTA') NOT NULL,
45     estado BOOLEAN DEFAULT TRUE
46 );
```

## 7 INTERFACES DO SISTEMA

Foram executados diversos testes para verificar o funcionamento do sistema.

### TELA DE LOGIN

Validação do acesso ao sistema.



The screenshot shows a web browser window displaying the login page for 'CLINICA NAKAMA'. The page is split into two main sections: a dark blue header on the left and a light blue login form on the right. The header contains the text 'CLINICA NAKAMA' and 'SISTEMA DE GESTÃO CLÍNICA' above a logo featuring a heart with a white cross and the text 'CLÍNICA NAKAMA' and '— CUIDANDO DA SUA SAÚDE —'. The login form on the right has a white silhouette of a person at the top. Below it are two input fields labeled 'EMAIL' and 'SENHA'. To the right of the 'SENHA' field is a checkbox labeled 'Ver Senha'. At the bottom of the form are two buttons: a red 'Cancelar' button and a green 'Entrar' button.

## TELA DE PACIENTES

| ID | NOME        | BILHETE         | NASCIMENTO | TELEFONE    | ENDEREÇO | HISTÓRICO      |
|----|-------------|-----------------|------------|-------------|----------|----------------|
| 1  | dadad       | 0008363881LA038 | 2000-01-01 | 2222222222  | dadad    | dadad          |
| 2  | Augusto PMD | 0008363881LA038 | 2003-03-03 | 935 166 548 | Mbemba   | Covid 19, SIDA |

## TELA DE CONSULTAS

Verificação de marcação, alteração e cancelamento.

| ID | PACIENTE    | MÉDICO     | DATA       | HORA     | ESTADO     |
|----|-------------|------------|------------|----------|------------|
| 2  | dadad       | Chaves     | 2000-06-01 | 14:11:11 | CONFIRMADA |
| 3  | Augusto PMD | Amen Kiata | 2028-06-17 | 20:00:00 | AGENDADA   |

Os resultados demonstraram que o sistema funciona de acordo com os requisitos definidos.

## 8 CRONOGRAMA DE DESENVOLVIMENTO

O cronograma abaixo ilustrado, descreve as actividades desenvolvidas ao longo de três semanas, periodo que exigiu muita dedicação e organização por nossa parte . Sendo assim nosso projecto foi desenvolvido com base o seguinte cronograma:

| PERÍODO | FOCO PRINCIPAL                               | ACTIVIDADES                                                                                                                                      |
|---------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 1º fase | Foi feita a estrutura e modelagem do sistema | Organização dos requisitos essenciais para o desenvolvimento do projecto                                                                         |
| 2º fase | Desenvolvimento do sistema                   | Desenvolvimento das telas principais do sistema, desenvolvimento das regras de negócio e validações                                              |
| 3º fase | Testes e entrega do sistema desenvolvimento  | Integração de todas as funcionalidades, tratamento das exceções de testes de integração, documentação técnica e preparação da apresentação final |

## CONCLUSÃO

O desenvolvimento do Sistema de Gestão de Clínica Médica permite aplicar conhecimentos fundamentais da Engenharia Informática, especialmente Programação Orientada a Objetos, Banco de Dados, Interface Gráfica e Arquitetura de software. A utilização dos padrões MVC e DAO desempenhou um papel fundamental no desenvolvimento do nosso Sistema de Gestão de Clínica Médica. A sua utilização permitiu organizar o código de uma forma eficiente, separar a lógica de acesso aos dados da lógica de negócios e garantir uma comunicação segura com o banco de dados MySQL.

Alem disso, vimos que a integração do DAO e MVC contribuiu para a construção de uma aplicação modular, escalável e de fácil manutenção, atendendo as boas práticas de desenvolvimento de software orientado a objetos.

Os resultados obtidos demonstraram que o sistema é capaz de atender às necessidades básicas de uma clínica médica, proporcionando o maior controlo, segurança e eficiência na gestão das informações.

## REFERÊNCIA BIBLIOGRÁFICA

Schildt, H. ([2019](#)). Java: The Complete Reference. McGraw-Hill.

Fowler, M. ([2002](#)). Patterns of Enterprise Application Architecture. Addison-Wesley.

Elmasri, R., & Navathe, S. ([2019](#)). Fundamentals of Database Systems. Pearson.

Deitel, P., & Deitel, H. ([2018](#)). Java: Como Programar. Pearson.

Horstmann, C. ([2021](#)). Core Java Volume I. Pearson.

DEITEL, P.; DEITEL, H. Java: Como Programar.

HORSTMANN, C. Core Java.

ELMASRI, R.; NAVATHE, S. Sistemas de Banco de Dados.

Documentação Oficial Java.

Documentação Oficial MySQL.